

Modelling the Life Cycle of the ModellingClub System in BPMN 2.0

Eight business processes covering maintenance, deployment, migration, data governance, security and the business life cycle of the full-stack platform built in Labs 4–6 — modelled as BPMN collaborations, with editable `.bpmn` sources.

Deliverables: this report (PDF) · 8 BPMN diagrams · 8 Camunda-compatible `.bpmn` files | Generated 2026-06-13

1 · Aim & scope

This report applies **BPMN 2.0** (Business Process Model and Notation, the OMG/ISO 19510 standard) to the **life cycle** of the *ModellingClub* system that was specified and built across the previous laboratories. Earlier labs delivered the running artefact: a Spring Boot + Angular community platform for RC / drone / scale-model builders (Lab 4), an RDF4J ontology of its domain (Lab 5) and a semantic (RDFa / JSON-LD) publication of its data (Lab 6). Those labs modelled what the software *does*. Lab 7 instead models the *processes that surround the software over its lifetime* — the activities a real deployment depends on but which are not, and need not be, encoded in the source: scheduled maintenance and backups, releases and deployments, a database migration, data retention and erasure, security incident response, dependency upgrades, the business contracts under which the system is hosted, and its eventual decommissioning.

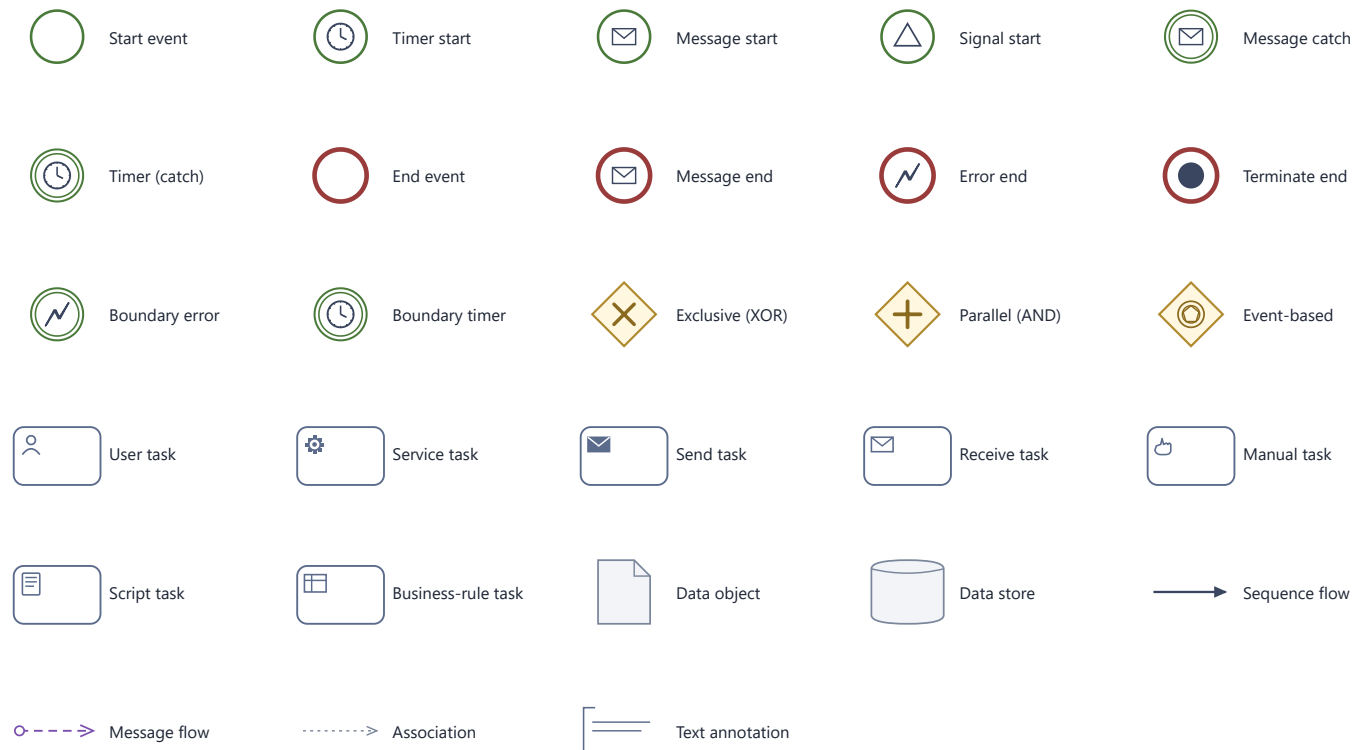
Crucially, these processes are governed by formal rules, good practice and external commitments rather than by code. Their participants reach well beyond "administrators and users": the eight models below introduce a DevOps engineer, a Release Manager, a DBA, a Data Protection Officer, a Security on-call engineer, an Operations Committee, an external Cloud Provider, a supervisory authority (GDPR regulator) and offsite storage. Modelling them in BPMN makes the responsibilities, hand-offs, decision points and failure/rollback paths explicit and auditable — exactly what a small platform needs before it can be operated responsibly in production.

2 · The system under study

ModellingClub is a full-stack web application. The backend is Spring Boot 3.5 on Java 21 (OpenAPI-first REST, JWT authentication, Spring AOP audit logging, JPA persistence); the frontend is Angular 21 with server-side rendering on a Node server. In development the database is an **H2** file (`modellingclubdb.mv.db`) and member-uploaded media (images, PDFs, STL files) live on disk under `uploads/`; the production roadmap migrates the database to **PostgreSQL** using Flyway and serves the stack behind nginx. The domain entities — Builds, Test runs, Marketplace listings, Messages, Telemetry, Reputation and Users (roles *member* / *admin*) — and real product features such as the account *data export* (JSON → ZIP), *account deletion*, the admin *moderation queue* and the 24-hour *JWT* tokens are the concrete anchors for the life-cycle processes that follow. Where a model references a command, a port or a configuration flag, it is the one actually used by the system (e.g. `mvn package`, `ddl-auto=validate`, the actuator health endpoint, the `/auth/login` route).

3 · BPMN notation used in this report

Each diagram is a BPMN **collaboration**: rectangular *pools* are the independent participants and the horizontal *lanes* inside a pool are roles/responsibilities. A solid arrow is a **sequence flow** (control flow inside one pool); a dashed arrow is a **message flow** (a message crossing the boundary between two pools). *Black-box* pools (drawn empty, marked ▪) are external participants whose internals are out of scope. Circles are **events** (thin border = start, thick = end, double = intermediate/boundary) and the icon inside gives the trigger (clock = timer, envelope = message, triangle = signal, bolt = error, ring = terminate). Diamonds are **gateways**: ✕ exclusive (choose one path on a condition), + parallel (all paths concurrently), and the pentagon is event-based. Rounded rectangles are **tasks**, with a small marker for the task type (person = user, gear = service, envelope = send/receive, hand = manual, page = script, table = business-rule). The legend below summarises the full palette used in this report.



Legend — the BPMN 2.0 element palette used across the eight models.

4 · Process catalogue

Process	Trigger	Participants (pools)	Key BPMN constructs
1. Scheduled Backup & Verification	Timer start event firing at 02:00 every day (cron scheduler).	ModellingClub Operations · Offsite Backup Storage (AWS S3) (external)	Timer start event, parallel gateway (split & join), exclusive (data-based) gateway, service/script/send/manual tasks, boundary error event, error end event, message flow to an external black-box pool, data object & data store.
2. Software Upgrade / Dependency Patch	Message start event: a Dependabot PR / CVE webhook arrives from the monitoring system.	ModellingClub Engineering · Monitoring System (external)	Message start event, business-rule task, exclusive gateways (severity branch + two outcome branches), intermediate catch timer (maintenance window), loop-back sequence flow, send tasks, message flows to/from an external pool, data objects, error end event.
3. Release & Blue-Green Deployment (CI/CD)	Message start event: a push / merge to the main branch reaches the CI pipeline.	ModellingClub CI/CD · Staging Environment (external) · Production Environment (external)	Message start event, parallel gateways (build & test split/join), exclusive gateways with loop-back, manual & user approval tasks, service & send tasks, message flows to external environment pools, data object & data store, multiple end events, rollback pattern.
4. H2 → PostgreSQL Database Migration	None start event: the DBA and Release Manager jointly approve the migration ticket.	ModellingClub Migration Team · Users (external) · Production Environment (external)	None start event, intermediate catch timer (maintenance window), exclusive gateways with loop-back, service/script/manual/user/send

Process	Trigger	Participants (pools)	Key BPMN constructs
			tasks, message flows to external Users & Production pools, data objects & data store, two end events, rollback path.
5. Data Archiving, Retention & GDPR Erasure	Two start events: a timer for the retention job and a message for a member GDPR request.	ModellingClub System · Member (external) · Archive Store (external)	Two start events (timer + message), exclusive gateways (request type, legal hold), boundary timer event (30-day deadline), send/service/script/manual tasks, message end events, message flows to Member & Archive Store pools, data store & data object.
6. Security Incident Response	Signal start event from Monitoring / SIEM (login spike, audit anomaly, or secret-leak alert).	ModellingClub Operations · Affected Users (external) · Supervisory Authority / Regulator (external)	Signal start event, three-way exclusive gateway (severity), parallel gateway (containment split/join), intermediate catch timer (72h breach deadline), boundary timer (escalation), send/service/manual/script tasks, message flows to Users & Regulator pools, post-incident gateway, data stores & object.
7. Member Onboarding & Hosting-Provider Provisioning	A prospective member submits the registration form; in parallel the committee starts a hosting capacity review.	Prospective Member · ModellingClub Operations · Cloud Provider	Collaboration with three pools, single-lane & multi-lane pools, message start event, send/receive & user/manual/service tasks, intermediate catch message events, boundary timer event, exclusive gateways, message flows crossing pools, data object & data store.

Process	Trigger	Participants (pools)	Key BPMN constructs
8. System Decommissioning / End-of-Life (EOL)	None start event: the Operations committee passes a formal EOL resolution.	ModellingClub Operations · Members · Cloud Provider	Multi-pool collaboration, terminate end event, two intermediate catch timers (notice & export windows), exclusive gateway (data to preserve?), parallel lanes, send/service/manual tasks, message flows to Members & Cloud Provider, data stores & object.

The eight processes are organised in four themes: (A) maintenance & operations, (B) deployment & migration, (C) data governance & security, (D) business life cycle.

1 · Scheduled Backup & Verification

Goal. Nightly automated backup of the ModellingClub database and the uploads/ media directory, with integrity verification, offsite transfer and a restore-from-last-good path on failure.

Trigger. Timer start event firing at 02:00 every day (cron scheduler).

Participants. ModellingClub Operations · Offsite Backup Storage (AWS S3) (external)

BPMN constructs. Timer start event, parallel gateway (split & join), exclusive (data-based) gateway, service/script/send/manual tasks, boundary error event, error end event, message flow to an external black-box pool, data object & data store.

Operational continuity for ModellingClub rests on a dependable backup. Every night at 02:00 a cron *timer start event* launches the job: a script task stamps the run, then a *parallel gateway* fans out two concurrent service tasks — one dumps the database (a file copy of `modellingclubdb.mv.db` in development, `pg_dump` under the planned PostgreSQL production profile), the other `tar + gzip`s the whole `uploads/` media directory. The streams re-join, and a script task computes SHA-256 checksums against a manifest — the integrity gate. An *exclusive gateway* branches on the result: valid checksums trigger a send task that ships the bundle to an external *black-box* Offsite Backup Storage pool (e.g. AWS S3) over a message flow, and the administrator is notified of success. A *boundary error event* on the transfer task catches network/authentication failures; that path, like a checksum mismatch, converges on an alert to the administrator, who is then walked through a manual restore from the last known-good bundle. The process closes on either a clean end event or an *error end event* that records the incident.

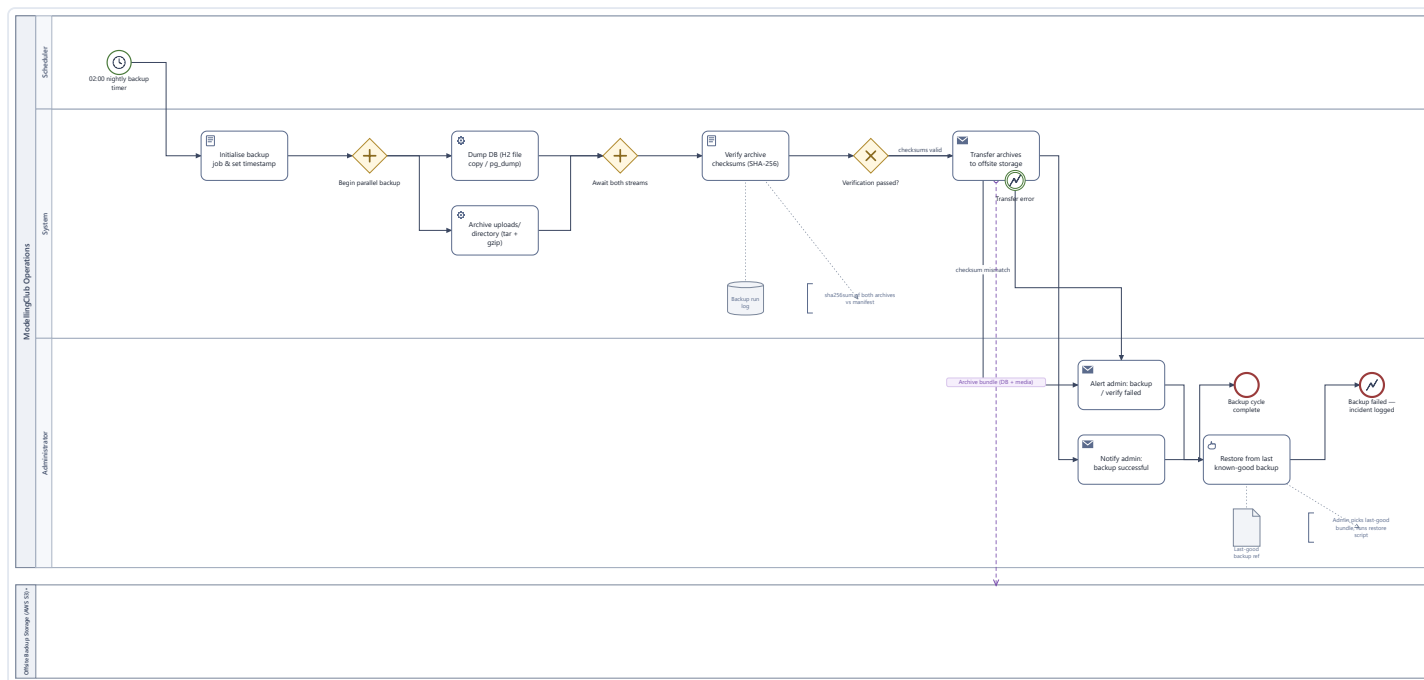


Figure 1 — Scheduled Backup & Verification (BPMN collaboration). Editable source: `bpmn/p1-backup.bpmn`.

2 · Software Upgrade / Dependency Patch

Goal. Manage a security / dependency update (e.g. a Dependabot PR for a Spring Boot or Angular CVE) from triage through staging validation, scheduled production deployment and rollback, so no unvetted code reaches production.

Trigger. Message start event: a Dependabot PR / CVE webhook arrives from the monitoring system.

Participants. ModellingClub Engineering · Monitoring System (external)

BPMN constructs. Message start event, business-rule task, exclusive gateways (severity branch + two outcome branches), intermediate catch timer (maintenance window), loop-back sequence flow, send tasks, message flows to/from an external pool, data objects, error end event.

Dependency hygiene is continuous. The process starts when the Monitoring pool delivers a Dependabot / CVE webhook as a *message start event*. A business-rule task scores the finding (CVSS, transitive dependency tree, Spring Boot / Angular release notes) and an *exclusive gateway* splits on severity: a critical vulnerability (CVSS ≥ 7.0 , e.g. a Spring RCE) takes an expedited track and mobilises on-call DevOps; a routine bump joins the ordinary patch queue. Both converge on a staging deployment that rebuilds the jar and the Angular SSR bundle and runs the integration suite. A staging gateway *loops back* to a developer fix task on failure, so the patch can be re-pinned and re-tested without ever touching production. Once staging is green, an *intermediate timer event* holds the flow until the agreed maintenance window (the next available slot for critical, Saturday 03:00 for routine). Production is updated with a rolling restart behind nginx; automated smoke tests then hit the actuator health endpoint, reporting back to Monitoring. A final gateway either notifies success or runs a rollback to the previous jar/bundle, signalling the incident and ending on an error end event.

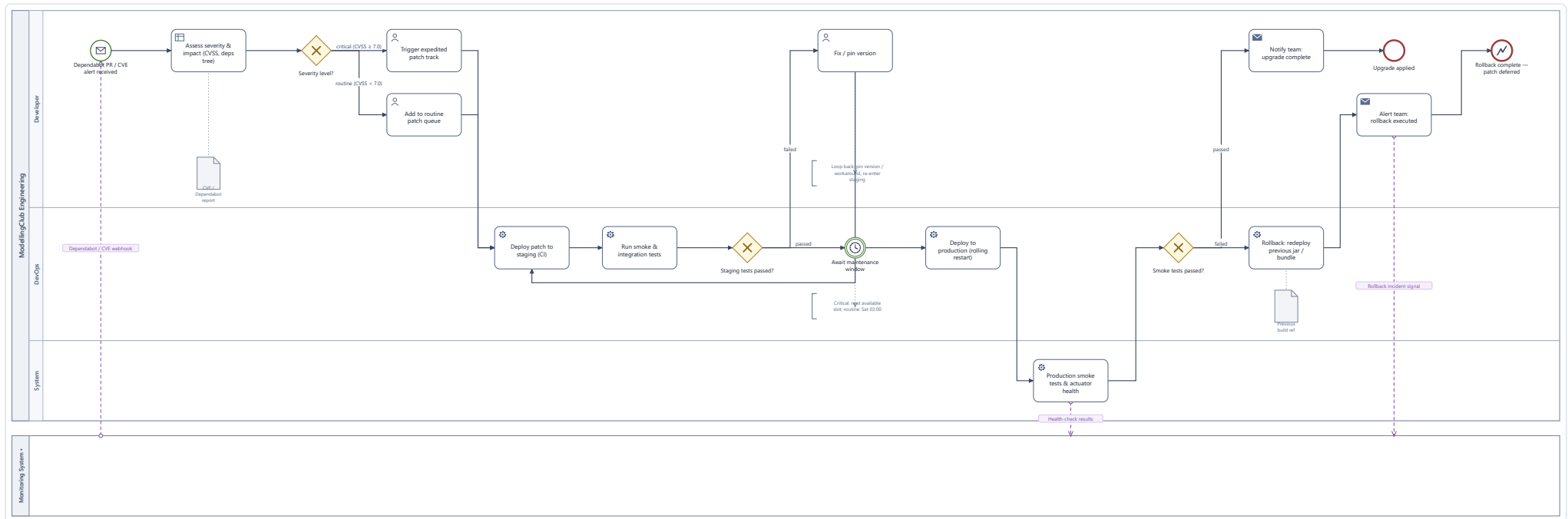


Figure 2 — Software Upgrade / Dependency Patch (BPMN collaboration). Editable source: [bpmn/p2-upgrade.bpmn](#).

3 · Release & Blue-Green Deployment (CI/CD)

Goal. Automate build, test, staging deploy and blue-green production deployment from a merge to main, with automatic rollback to the previous (blue) release if health checks or smoke tests fail.

Trigger. Message start event: a push / merge to the main branch reaches the CI pipeline.

Participants. ModellingClub CI/CD · Staging Environment (external) · Production Environment (external)

BPMN constructs. Message start event, parallel gateways (build & test split/join), exclusive gateways with loop-back, manual & user approval tasks, service & send tasks, message flows to external environment pools, data object & data store, multiple end events, rollback pattern.

This is the everyday change pipeline. A merge to `main` arrives as a *message start event* into the CI Pipeline lane and immediately fans out through a *parallel gateway*: Maven builds the backend jar while Angular bundles the SSR frontend. The artifacts re-join and a second parallel split runs the test suites concurrently (`mvn test`; `npm test` + `tsc --noEmit`). A test gateway *loops failures back* to the developer to fix and re-run. On success the artifacts are published and auto-deployed to a *Staging* black-box pool, where QA runs a manual smoke test; failures rejoin the fix loop. A passing build reaches a *manual approval gate* held by the Release Manager, who may reject (ending the run) or approve. Approval triggers a **blue-green** deployment: Ops deploys to the idle green slot of the *Production* pool while blue keeps serving, runs an actuator health check, then switches the nginx upstream and runs a post-deploy smoke test. Any failure in the health check or post-deploy smoke takes the *rollback* path — nginx reverts to the still-running blue slot, guaranteeing zero downtime — while success ends the process cleanly. The five lanes (Developer, CI Pipeline, QA, Release Manager, Ops) make every hand-off explicit.

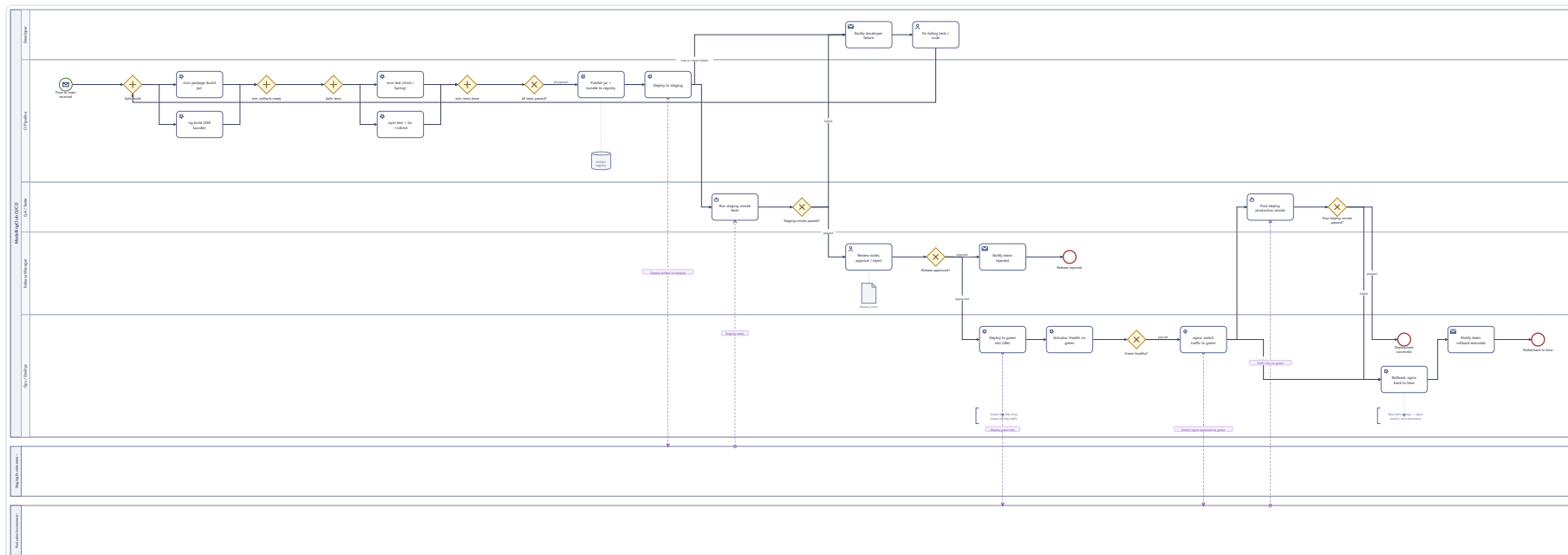


Figure 3 — Release & Blue-Green Deployment (CI/CD) (BPMN collaboration). Editable source: [bpmn/p3-cicd.bpmn](#).

4 · H2 → PostgreSQL Database Migration

Goal. Execute a one-time, audited cutover of the H2 file database to PostgreSQL in production, with a Flyway baseline, data migration, verification and a tested rollback to H2.

Trigger. None start event: the DBA and Release Manager jointly approve the migration ticket.

Participants. ModellingClub Migration Team · Users (external) · Production Environment (external)

BPMN constructs. None start event, intermediate catch timer (maintenance window), exclusive gateways with loop-back, service/script/manual/user/send tasks, message flows to external Users & Production pools, data objects & data store, two end events, rollback path.

A planned, reversible cut-over from H2 to PostgreSQL — the migration described in the project runbook. After the DBA and Release Manager approve (a *none start event*), the developer adds the Postgres driver and Flyway, and the DBA generates `V1__baseline.sql` from the H2 schema, hand-adapting dialect differences (UUID and `TIMESTAMPZ` types). Testcontainers integration tests validate the mappings against a real Postgres image; failures *loop back* to a SQL-fix task. Once green, the Release Manager schedules a maintenance window and sends a notice to the external *Users* pool; an *intermediate timer event* holds the flow until the window opens. Ops then puts the live system into maintenance/read-only mode (a message to the *Production* pool), the DBA dumps H2 in FK-safe order, provisions Postgres, loads the dump and runs `flyway migrate`, and the datasource is switched with `ddl-auto=validate`. Verification queries (row counts, FK integrity) and a QA smoke test each gate the outcome: either maintenance mode is lifted and the Release Manager signs off and archives the runbook, or — because the H2 file is left untouched throughout — the datasource is reverted to H2 and the team is notified. Two end states make the rollback path first-class.

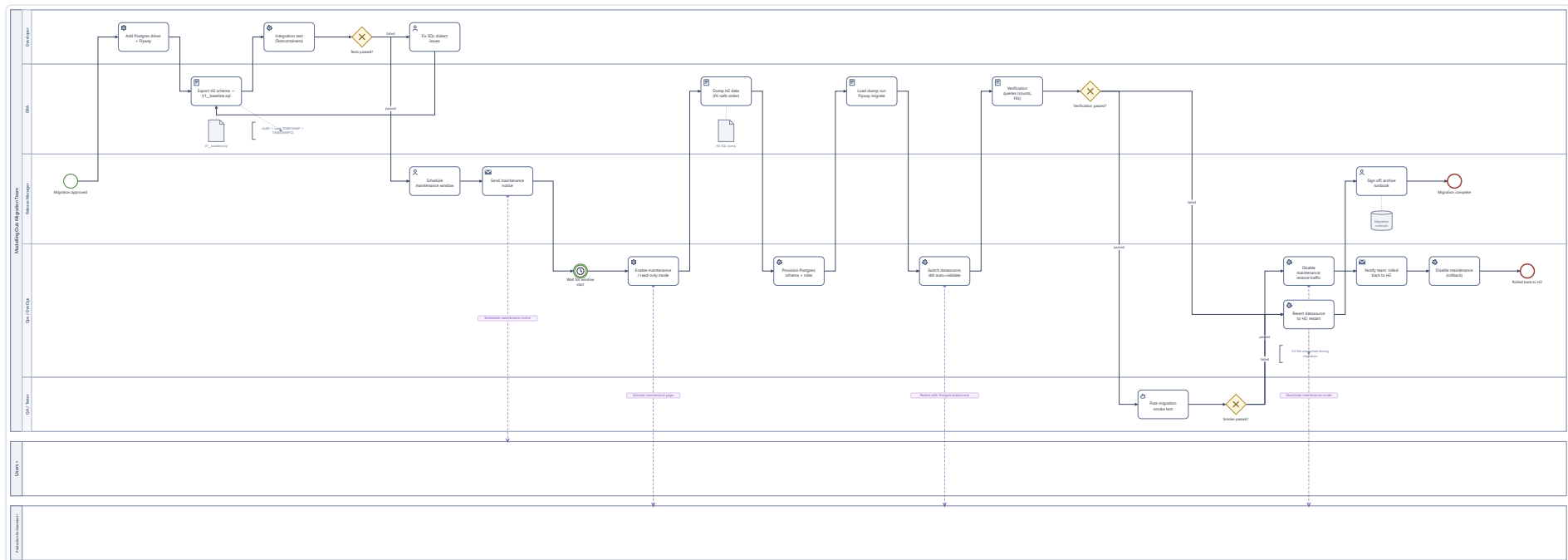


Figure 4 — H2 → PostgreSQL Database Migration (BPMN collaboration). Editable source: [bpmn/p4-migration.bpmn](#).

5 · Data Archiving, Retention & GDPR Erasure

Goal. Periodically archive and purge stale operational data to cold storage, and fulfil member-initiated GDPR requests (data export or right-to-erasure) within statutory deadlines.

Trigger. Two start events: a timer for the retention job and a message for a member GDPR request.

Participants. ModellingClub System · Member (external) · Archive Store (external)

BPMN constructs. Two start events (timer + message), exclusive gateways (request type, legal hold), boundary timer event (30-day deadline), send/service/script/manual tasks, message end events, message flows to Member & Archive Store pools, data store & data object.

One model carries two intertwined data-governance duties, with two start events. A nightly *timer start* runs the retention job: it identifies records past their retention window (old test runs, messages, sold listings), archives the batch to an external *Archive Store* pool, purges them from the live database and writes an audit entry. Independently, a *message start* fires when a member submits a request from their account page. Identity is verified against the JWT session and the username challenge already enforced in the UI, and an *exclusive gateway* branches on request type. An *export* compiles the member's profile, builds and comments into the JSON → ZIP download already offered by the product, delivered over a message flow. A *deletion* (right to erasure) goes to the DPO lane: if a legal hold applies (an open dispute or financial-record obligation) the request is parked and the member notified; otherwise marketplace transaction records are anonymised in place for compliance, the remaining personal data is hard-deleted, and a confirmation is sent. A *non-interrupting boundary timer* on identity verification enforces the GDPR Article 12 thirty-day deadline, escalating to notification if the handler stalls. Message end events mark the delivered export and completed erasure.

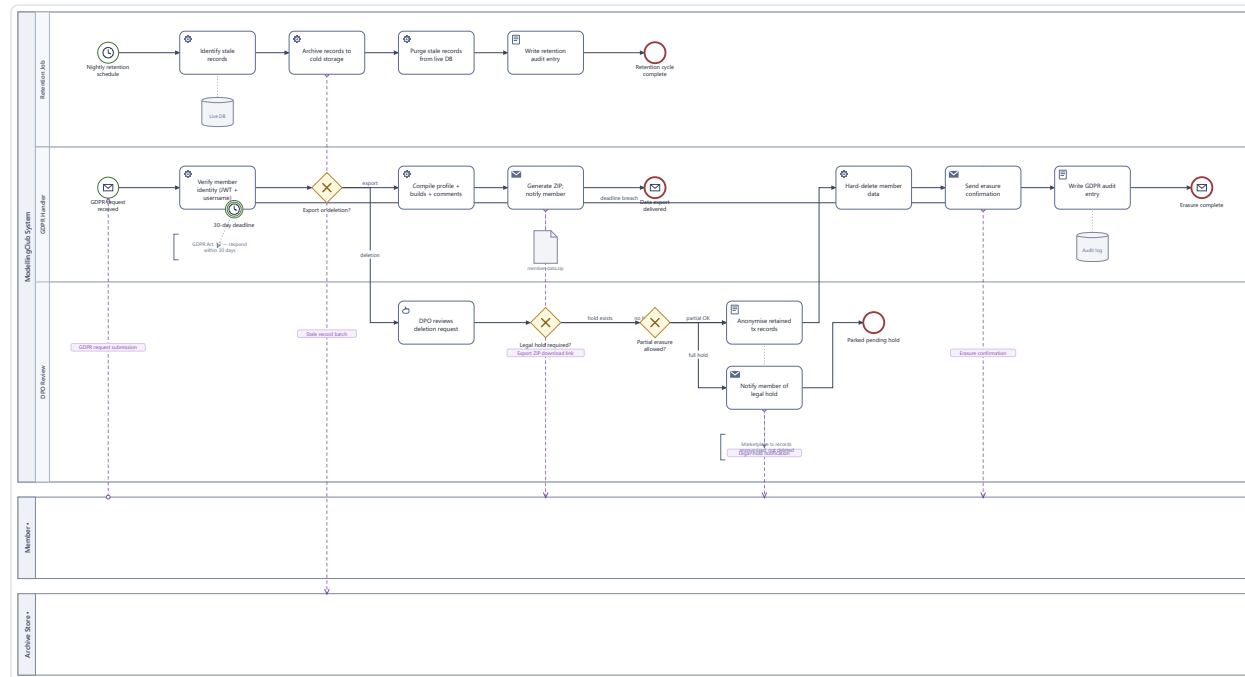


Figure 5 — Data Archiving, Retention & GDPR Erasure (BPMN collaboration). Editable source: [bpmn/p5-data-gdpr.bpmn](#).

6 · Security Incident Response

Goal. Detect, triage, contain, eradicate and recover from security incidents (brute-force login, leaked JWT secret, abnormal admin activity), with GDPR breach notification where personal data is exposed.

Trigger. Signal start event from Monitoring / SIEM (login spike, audit anomaly, or secret-leak alert).

Participants. ModellingClub Operations · Affected Users (external) · Supervisory Authority / Regulator (external)

BPMN constructs. Signal start event, three-way exclusive gateway (severity), parallel gateway (containment split/join), intermediate catch timer (72h breach deadline), boundary timer (escalation), send/service/manual/script tasks, message flows to Users & Regulator pools, post-incident gateway, data stores & object.

When the SIEM crosses a threshold it raises a *signal start event* — for example a brute-force spike on the un-rate-limited `/auth/login` route, an anomaly in the Spring AOP audit log, or an alert that the HS256 JWT secret has leaked. An enrichment task correlates logs, IP reputation and affected accounts before a three-way *exclusive gateway* triages severity. *Low* events are logged and thresholds tightened; *high* events get scoped mitigations and a team notification; *critical* events enter a *parallel* containment block that rotates the JWT secret (which, by design, invalidates every 24-hour token and forces a global re-login), blocks the offending IPs, and optionally engages maintenance mode. A *non-interrupting boundary timer* escalates a stalled high-severity case to critical after one hour. After containment re-joins, a breach gateway decides whether personal data was exposed; if so the DPO is engaged, affected users are notified, and a *72-hour intermediate timer* enforces the GDPR Article 33 obligation to notify the supervisory authority. The process always ends with eradication, recovery verification and a mandatory post-incident review whose gateway can spawn a concrete hardening task — for instance, finally adding rate limiting to `/auth/login` — so incidents drive durable improvement.

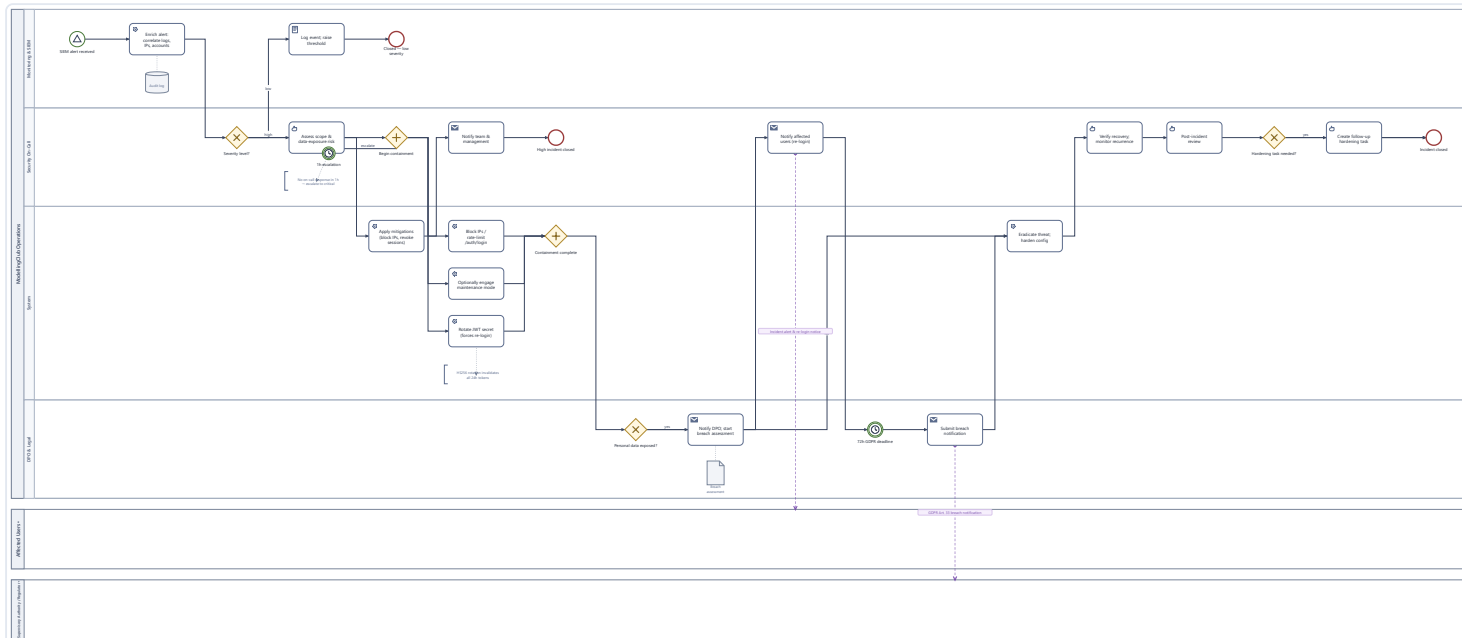


Figure 6 — Security Incident Response (BPMN collaboration). Editable source: `bpmn/p6-incident.bpmn`.

7 · Member Onboarding & Hosting-Provider Provisioning

Goal. Formalise a new member's entry into ModellingClub while the Operations committee negotiates and activates the hosting / SLA contract with a cloud provider — a three-party collaboration.

Trigger. A prospective member submits the registration form; in parallel the committee starts a hosting capacity review.

Participants. Prospective Member · ModellingClub Operations · Cloud Provider

BPMN constructs. Collaboration with three pools, single-lane & multi-lane pools, message start event, send/receive & user/manual/service tasks, intermediate catch message events, boundary timer event, exclusive gateways, message flows crossing pools, data object & data store.

A three-party collaboration covering both the human and the organisational side of growth. A *Prospective Member* submits the registration form, which crosses as a message to the Membership Admin lane; the admin creates a pending account and sends a JWT-encoded verification email. A *boundary timer* resends once if the link is not clicked within 24 hours; an *exclusive gateway* then either activates the account (after the admin moderates the member's first build, mirroring the real admin queue) or lets the registration expire. In parallel — and this is the part not visible in the source code — the *Operations Committee* negotiates the infrastructure the platform runs on: it sends a hosting quote request to an external *Cloud Provider* pool, receives a quote and SLA, and an exclusive gateway accepts or rejects on cost and uptime terms (e.g. reject if uptime < 99.5%). On acceptance the signed contract is transmitted, the provider provisions the environment and signals readiness, and the committee configures DNS and goes live. The intermediate message catch events and the cross-pool message flows (registration, verification, quote, signed contract, environment-ready) make this a textbook BPMN collaboration with genuinely independent, communicating participants.

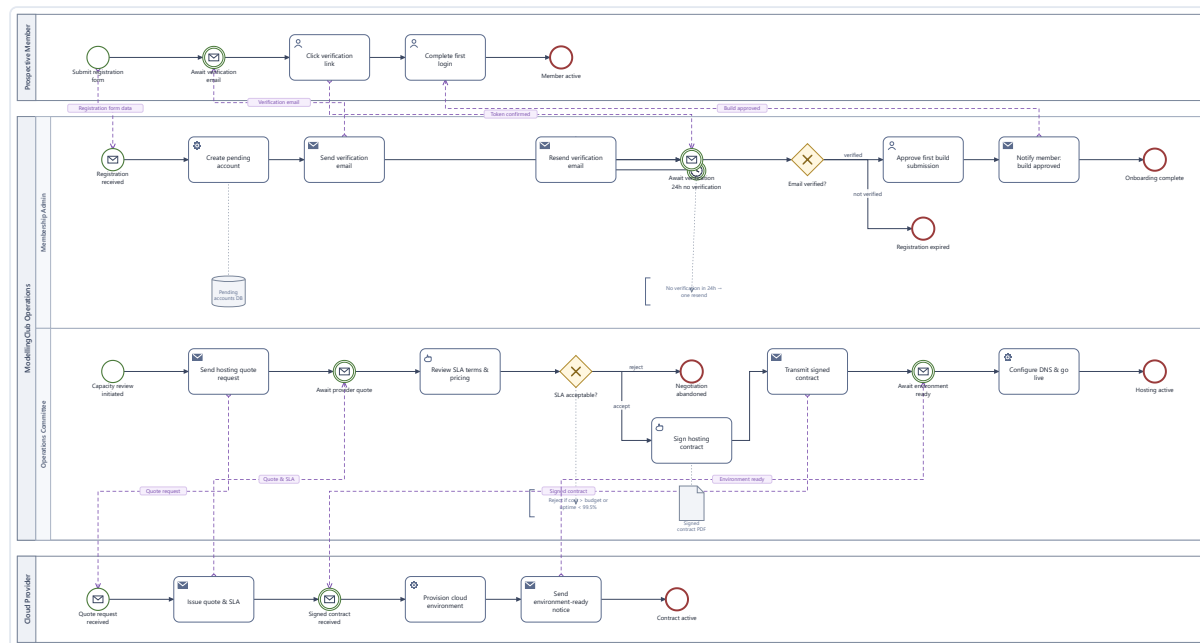


Figure 7 — Member Onboarding & Hosting-Provider Provisioning (BPMN collaboration). Editable source: [bpmn/p7-onboarding.bpmn](#).

Trigger. None start event: the Operations committee passes a formal EOL resolution.

Participants. ModellingClub Operations · Members · Cloud Provider

BPMN constructs. Multi-pool collaboration, terminate end event, two intermediate catch timers (notice & export windows), exclusive gateway (data to preserve?), parallel lanes, send/service/manual tasks, message flows to Members & Cloud Provider, data stores & object.

Participants. ModellingClub Operations · Members · Cloud Provider

foreware has an end. When the Operations Committee passes an EOL resolution (a *none start event*), it announces the shutdown to the external *Members* pool and waits out the 30-day statutory notice period on an *intermediate timer*. It then opens a final data-export window — reusing the product's own JSON → ZIP export so members can retrieve their builds and media — held open by a second timer while the Membership Admin lane supports requests. In parallel, the resolution signals the Legal & DPO lane, which reviews retention obligations and moves records that must survive (financial / contest data, kept per law for ~7 years) to a long-term legal archive. After the window closes and the platform is locked, an *exclusive gateway* decides whether any data must be migrated or handed over before a full archive is written to cold storage; all keys, tokens and TLS credentials are revoked; and a formal termination notice crosses to the *Cloud Provider*, which tears down the VPS, storage and DNS and confirms. A final public notice is published and the process closes on a *terminate end event*, cancelling any remaining tokens to mark the system fully decommissioned.

5 · Correctness & deliverables

Syntactic and logical correctness. Every model was checked against the BPMN execution semantics: each path reaches an end event; every gateway either re-merges its branches or each branch terminates; outgoing flows of exclusive and event-based gateways carry mutually-exclusive conditions; parallel splits are matched by parallel joins; message flows only ever cross a pool boundary and never connect two nodes of the same pool. In addition to the rendered diagrams in this report, each process is delivered as an editable `.bpmn` file (BPMN 2.0 XML with diagram interchange) that opens directly in the Camunda Modeler; all eight files were verified to import without warnings using the `bpmn-moddle` reference parser that underpins Camunda / bpmn.io.

Deliverables in this submission

- `lab7-report.pdf` — this report (textual description + all eight BPMN diagrams).
- `bpmn/*.bpmn` — eight editable BPMN 2.0 files (open in Camunda Modeler).
- `diagrams/*.svg` — the rendered diagrams as standalone vector images.
- `src/`, `scripts/` — the generator that produces the diagrams and this report from a single model.

References

1. **OMG.** Business Process Model and Notation (BPMN) Version 2.0.2. <https://www.omg.org/spec/BPMN/>
2. **ISO/IEC.** 19510:2013 — Information technology — OMG BPMN. <https://www.iso.org/standard/62652.html>
3. **Camunda.** BPMN 2.0 Modeling Reference. <https://camunda.com/bpmn/reference/>
4. **Camunda.** BPMN 2.0 Examples. <https://camunda.com/bpmn/examples/>
5. **Camunda.** Modeler (desktop BPMN editor). <https://camunda.com/products/modeler/>
6. **BPMB.** BPMN 2.0 Poster / cheat-sheet. http://www.bpmb.de/images/BPMN2_0_Poster_EN.pdf
7. **EU.** GDPR — Regulation (EU) 2016/679, Articles 12, 17, 33. <https://gdpr-info.eu/>